

Extensibility is the companion principle to reusability. A reusable specification must not be rigid. It should evolve as products grow, regulations change, architectures modernise, and AI tools become more capable. Extensible SDD depends on modular specifications, version control, traceability, and structured change impact analysis. A modular specification separates stable intent from variable implementation details. For example, a payment workflow specification may keep the core business rules stable while allowing extensions for new payment providers, fraud checks, reporting obligations, or regional compliance requirements.

Versioning is essential for this evolution. Teams should know which version of a specification produced which implementation, which acceptance criteria were valid at the time of release, and what changed when a new requirement was introduced. Traceability links requirements to design decisions, code changes, tests, approvals, and operational evidence. This makes it possible to assess the impact of change before agents modify code.

Reusable and extensible SDD reduces duplication, improves governance, accelerates delivery, and helps enterprises scale AI-assisted software development without losing

control over quality or intent.

Developing framework-agnostic SDD

Framework-agnostic SDD means designing specifications so that the expression of intent is independent of a specific programming language, software framework, cloud provider, AI coding assistant, issue tracker, or DevOps platform. The central idea is to separate what the system must achieve from how a particular team chooses to implement it. This distinction is critical in modern enterprises because the same business capability may be implemented across different technology stacks such as Java and Spring Boot, .NET, Python, Node.js, React, native mobile applications, microservices, data pipelines, Kubernetes workloads, serverless functions, or legacy platforms. If the specification is tightly coupled to one framework, it loses value when the implementation environment changes.

A framework-agnostic SDD approach begins with stable engineering artifacts. These include intent, scope, requirements, non-goals, assumptions, constraints, interfaces, data contracts, quality attributes, acceptance criteria, verification strategy, and governance gates. These elements should remain understandable

even if the implementation moves from a monolith to microservices, from on-premises infrastructure to cloud-native deployment, or from one AI coding platform to another. For example, a requirement such as “the system must validate customer identity before initiating a funds transfer” is framework-agnostic. The implementation may differ across Java, .NET, or Python, but the business rule, verification expectation, audit requirement, and security constraint remain the same.

Modern technical standards make framework-agnostic SDD more practical. OpenAPI can describe REST interfaces independently of implementation language. AsyncAPI can define event-driven contracts across messaging systems. JSON Schema can formalise data structures. C4 model diagrams can describe architecture at different levels of abstraction. Architecture Decision Records can capture important design choices without embedding them only in code. Policy-as-code tools can express security, compliance, and infrastructure rules in a repeatable way. Test contracts can define expected behaviour that must pass regardless of the underlying stack. CI/CD evidence can then prove that the implementation satisfies the specification in a specific environment.

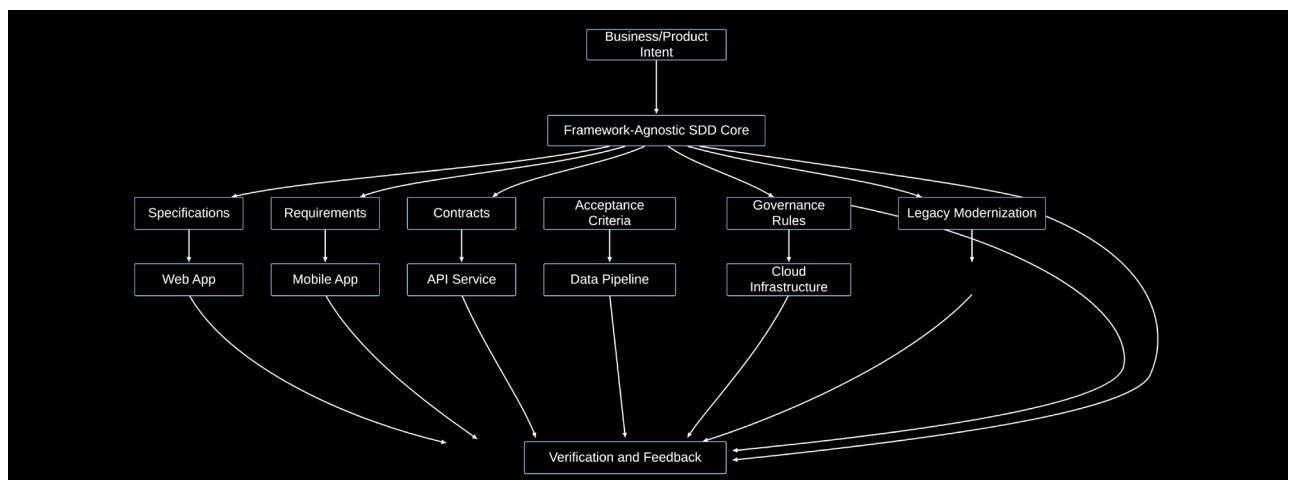


Figure 2: Developing framework-agnostic SDD